湖南科技大学计算机科学与工程学院

智能终端系统及应用开发课程设计报告

专业班级： 24 级物联网工程三班

姓　名： 李思民

学　号： 2405040303

指导教师： 黄晶

时　间： 2026.10.5~2026.10.26

地　点： 逸夫楼 433

指导教师评语：

成绩：　　**等级：**

签名：

目录

一　实验题目.....3
二　实验目的.....3
三　总体设计.....3
3.1 需求分析与功能边界.....3
3.2 技术方案与模块划分.....4
3.3 系统架构与数据流.....4
四　详细设计.....5
4.1 主界面导航与组件路由.....5
4.2 首页浏览与内容发现.....6
4.3 网络请求与首页分页.....7
4.4 视频播放与生命周期管理.....9
4.5 用户登录与互动状态.....12
4.6 广场与内容分类.....13
4.7 本地浏览记录与数据持久化.....14
五　实验结果与分析.....16
5.1 功能验证与测试设计.....16
5.2 运行现象与结果分析.....16
5.3 问题分析与改进方向.....17
5.4 需要补充的运行证据.....17
六　小结与心得体会.....18

一　实验题目

本项目基于 Android 的组件化短视频应用设计与实现。项目工程名称为 shortVideo，围绕视频内容浏览与播放，组织首页、广场、发现和个人中心四个主要入口，形成“用户登录—浏览内容—进入播放—互动收藏与历史回看”的使用流程。

客户端采用 Java 与 Kotlin 混合开发，使用 MVVM、DataBinding 和组件路由组织页面，通过 Retrofit 与 OkHttp 对接既有远程接口，并利用 Room 保存本地浏览记录。设计重点为 Android 客户端的功能组织、数据交互和媒体资源管理，服务端接口由现有服务提供。

二　实验目的

1. 掌握 Android 多页面应用的组织方法，将 Activity、Fragment、ViewPager2 和 RecyclerView 综合应用于内容浏览场景，理解页面切换、列表复用和生命周期之间的关系。

2. 理解 MVVM 中界面、状态与数据访问的职责，使用 LiveData 与 DataBinding 传递数据，减少界面代码与网络逻辑的直接耦合。

3. 掌握组件化工程的基本结构，利用 ARouter 完成跨模块页面跳转和参数传递，认识公共基础模块在复用和统一交互中的作用。

4. 掌握视频播放的基础流程，使用 Media3 ExoPlayer 加载媒体地址，并根据页面生命周期暂停与释放资源，避免后台播放及资源泄漏。

5. 掌握远程数据与本地持久化的协同方式，理解分页、登录令牌、浏览记录、异步数据库操作及错误状态处理。

6. 形成以证据验证功能的习惯，对正常流程、异常输入、断网及登录状态变化分别设计测试，分析当前实现的不足。

7. 本设计以“内容获取、界面展示、媒体播放、用户互动和数据保存”为学习主线，在实现功能的同时关注模块边界、异步操作与状态一致性。

三　总体设计

3.1 需求分析与功能边界

应用面向通过移动终端浏览短视频的用户。游客主要浏览推荐、日报、分类和视频详情；登录后可使用点赞、收藏、评论和个人资料管理。首页采用纵向视频卡片列表，点击后进入独立播放页，并非全屏上下滑动切换的视频流。微信登录、微信分享及视频拍摄发布链路不列入已完成功能，相册模块主要服务于头像更换。

核心使用场景分为三类：游客浏览内容并播放视频；登录用户进行收藏、点赞和评论；用户通过收藏与本地记录回到已关注的内容。用户操作应能够获得明确反馈，网络不可用时保留必要的返回与重试入口。

3.2 技术方案与模块划分

工程以 app 为集成入口，业务模块依赖公共基础能力，通过路由访问其他业务页面。根工程 isModule=false 时，各业务模块以库的形式参与集成；切换组件模式时，公共构建脚本调整插件与清单文件，以支持模块调试。

系统各模块的划分及主要职责见表 3-1。

表 3-1　系统模块划分与主要职责

模块	主要职责
app	应用启动、底部四入口、Fragment 切换
library_base	页面基类、路由常量、用户管理、Room、工具类
network 与 video_data	Retrofit 请求封装、统一响应和业务数据模型
feature_home	推荐与日报容器，复用视频列表
feature_mediaplay	播放、互动、搜索、分类列表、收藏、浏览记录
feature_piazza 与 feature_find	广场图文、图片浏览、分类、主题与话题入口
feature_user 与 feature_camera	验证码登录、资料设置、头像拍照

工程编译配置为 compileSdk 35、targetSdk 35、minSdk 24，Java 编译目标为 11。核心依赖包括 Retrofit 2.11.0、OkHttp 4.12.0、Glide 4.16.0、Room 2.6.1、ARouter 1.5.2 和 Media3 1.0.0。版本描述以工程配置为准，用于说明本项目采用的开发环境。

3.3 系统架构与数据流

系统总体架构如图 3-1 所示。业务模块通过路由协作，公共层集中提供基础能力；远程接口与本地数据库承担不同的数据职责。图中表示职责关系，不表示所有业务模块之间存在直接编译依赖。

远程接口负责视频内容、用户资料和互动数据。用户管理器使用 EncryptedSharedPreferences 保存令牌和必要资料；浏览记录使用 Room 单独保存。视频媒体仍通过播放地址在线加载，浏览记录只保存索引与展示字段，不等于离线视频缓存。

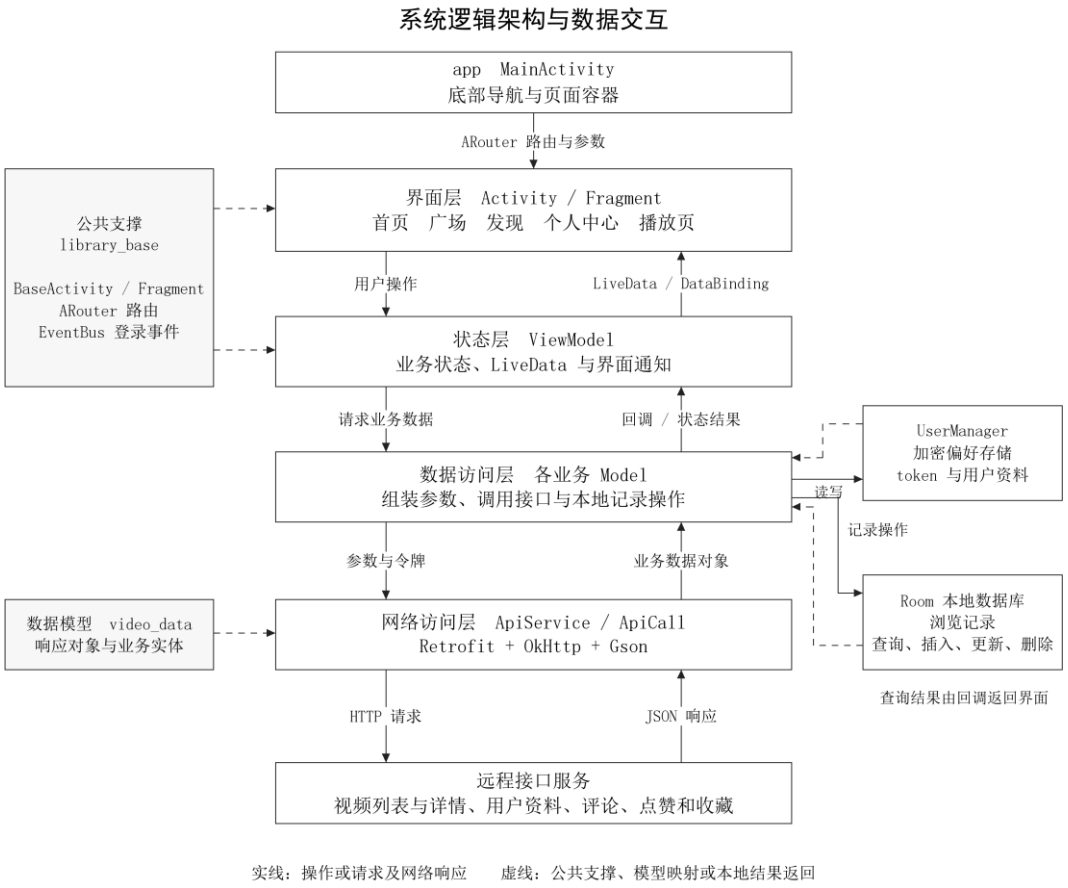


图 3-1 系统总体架构

主要数据流为：页面发起操作，ViewModel 组织状态，Model 请求 API，再将结果转换为 LiveData 更新界面。登录后广播登录事件，相关页面重新获取资料或互动状态，使个人中心与播放页保持一致。

视频列表对象 ResVideo 提供卡片展示字段，ArchivesInfo 提供播放地址、正文及互动状态，ResVideoAllInfo 聚合详情相关数据。列表项只传递编号，播放页再读取完整详情，可避免把大对象随页面跳转反复传递。

四 详细设计

4.1 主界面导航与组件路由

MainActivity 将首页、广场、发现和我的入口映射到路由路径。首次切换时创建并添加对应 Fragment，后续以 show 和 hide 切换。该方式能够保留列表位置与页面数据，适合四个入口间频繁切换的场景；页面保留也意味着需要注意内存占用以及隐藏页面的后台工作。

路由常量集中定义在 ArouterPath 中，视频列表通过视频编号跳转播放页，目标页面通过参数注入获得编号，再请求完整详情。调用方不必直接引用目标 Activity 的具体类型，从而降低模块之间的耦合。

代码 4-1　视频列表跳转播放页的调用方式

```
ARouter.getInstance()
    .build(ArouterPath.Video.ACTIVITY_MEDIA_PLAY)
    .withInt(ArouterPath.Video.KEY_VIDEO_ID, videoId)
    .navigation();
```

代码来源：VideoListFragment.java 的路由调用方式，参数变量作简化（节选）。

路径常量确定目标页面，KEY_VIDEO_ID 约定双方使用同一参数名称。页面收到编号后请求详情，路由因此连接的是页面入口，而不是代替业务接口完成数据查询。编号无效或详情为空时，还需要界面提示并停止播放初始化。

以上为路由调用方式的简化示意。工程公共基类承担 ViewModel 初始化、DataBinding 绑定和参数注入，并统一观察加载、提示及结束页面等状态，使业务页面集中处理自身逻辑。

底部导航使用 add、show 和 hide 保留 Fragment。保留页面可以减少重复请求，但隐藏页面仍处于工程管理之中，不能把“不可见”等同于“已销毁”。媒体等持续占用资源的行为应由播放页生命周期单独管理。

4.2 首页浏览与内容发现

首页通过“推荐”和“日报”切换视频列表，发现页通过分类、主题和话题组织内容。两者分别服务于连续浏览与定向查找。图 4-1 展示了现有运行截图中的界面结构。

首页卡片包含封面、标题和作者信息。封面使用 Glide 异步加载，列表使用 RecyclerView 回收条目视图；点击卡片进入播放页。发现页的分类与主题入口将浏览范围缩小到相应内容集合。截图能够证明页面布局，分页与各入口的运行稳定性仍需按测试用例验证。

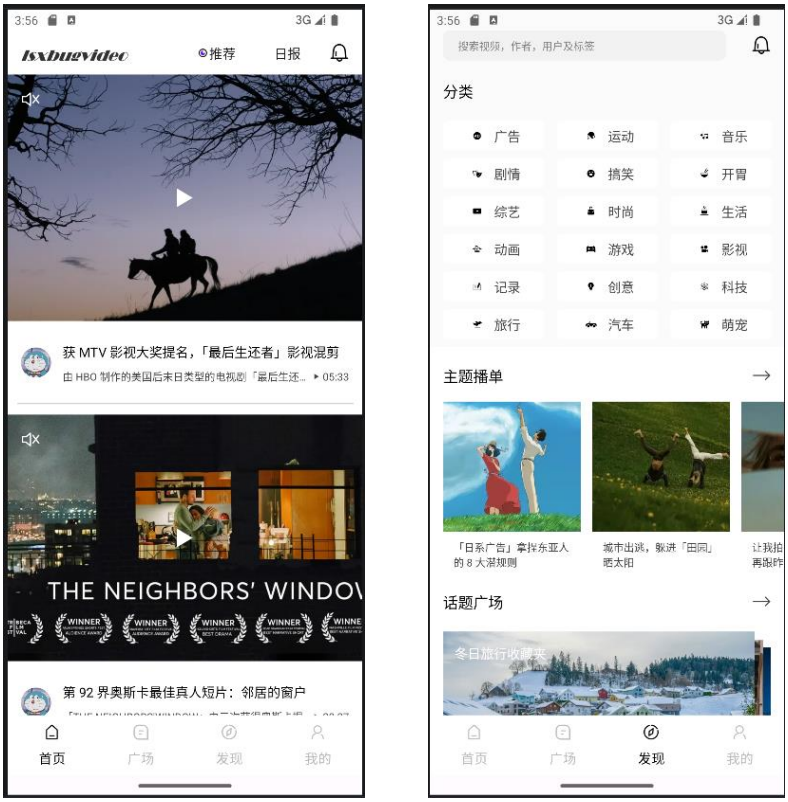


图 4-1　首页与发现界面

（a）首页推荐列表　　（b）发现分类与主题

4.3 网络请求与首页分页

网络层采用 Retrofit 定义接口、OkHttp 执行请求、Gson 解析 JSON。Java 回调封装与 Kotlin 协程封装目前并存，播放页采用 Kotlin 链路。统一响应包含 code、msg、time 和 data，列表数据进一步包含 count 与 list。各接口需要根据实际业务码判断成功，不能仅凭 HTTP 请求完成就视为业务成功。

首页由 ViewPager2 承载推荐和日报两页，列表复用 VideoListFragment。首次加载与下拉刷新从第一页请求，上拉加载请求后续页；刷新替换旧数据，加载更多追加数据。当前列表按每页十条组织，不足一页时关闭继续加载。异常处理需保留已有列表，并结束刷新动画，避免用户误以为请求仍在进行。

分页请求与列表更新流程如图 4-2 所示。

当前 VideoListModel 在请求前自增页码，失败回调仅向上层传递错误。若某页失败后再次加载仍继续自增，可能跳过失败页。因此，改进时应保存本次目标页，成功后再提交页码，或者失败时回退，同时限制同一列表的并行加载。

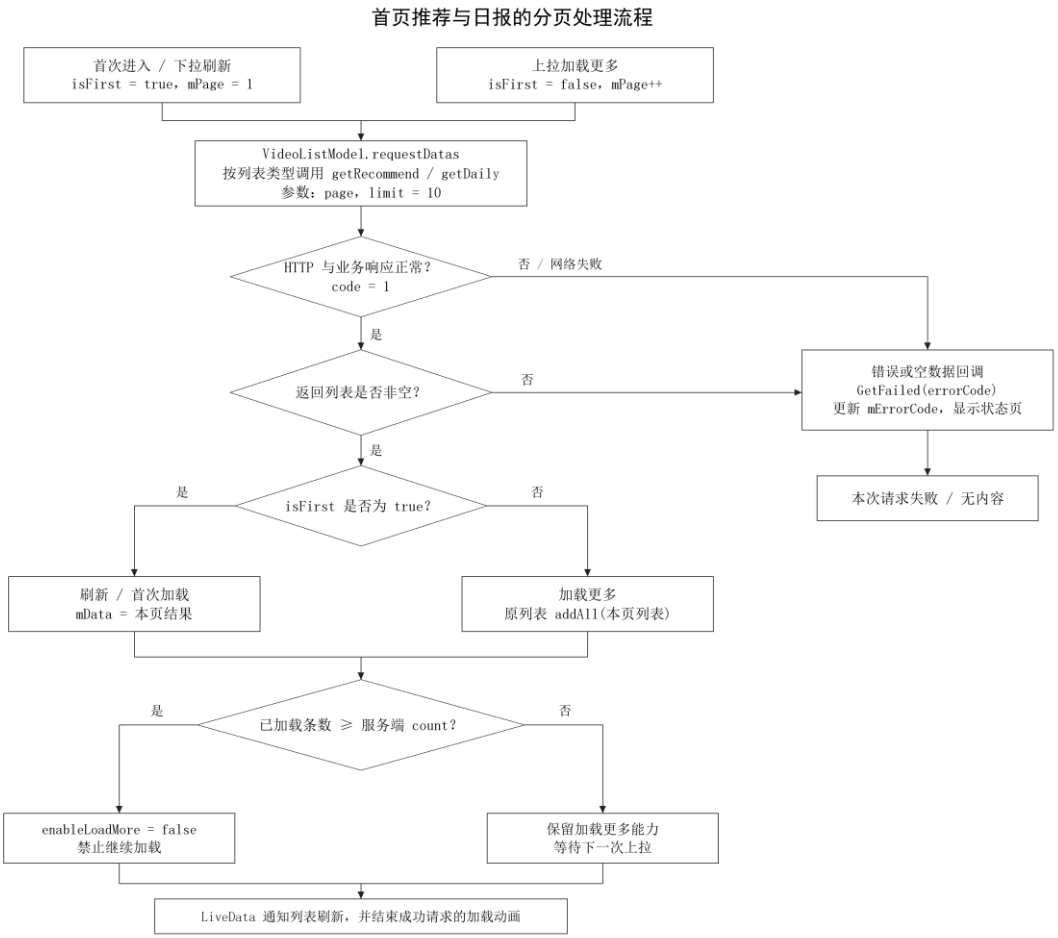


图 4-2　分页请求与列表更新流程

4.3.1 请求状态的统一处理

代码 4-2　统一封装对业务结果的判断

```
val response = call()
if (response.getCode() == successCode) {
    val data = dataType(response)
    when {
        data != null -> NetworkStatusResult.Success(data)
        else -> NetworkStatusResult.Empty(response.getMsg())
    }
} else {
    NetworkStatusResult.Error(response.getMsg())
}
```

代码来源：network/.../ApiCallKotlinVersion.kt（节选）。

封装通过 successCode 判断业务成功，再利用 dataType 提取具体数据，将界面接收的结果归纳为 Success、Empty 和 Error。这样，页面不必重复解析响应外壳；但 Empty 的具体含义仍由业务接口决定，例如无返回实体的操作与空列表并不完全相同。

该封装对 CancellationException 原样抛出，使协程取消继续传播；其余异常转换为网络错误结果。ViewModel 使用 viewModelScope 发起请求，生命

周期结束时可取消协程。请求完成不等于媒体资源自动释放，播放器仍需要独立管理。

不同请求状态下的界面处理与核对要点见表 4-1。

表 4-1　网络请求状态与界面处理

场景	界面处理与核对点
成功返回列表	刷新替换或加载追加；检查是否重复
空列表	区分首次无内容和已经到达末页
网络或业务失败	结束加载状态，保留已有内容并提示原因
登录令牌失效	清理或更新会话，引导重新登录

4.4 视频播放与生命周期管理

实际路由指向 MediaPlayActivityKotlinVersion。播放页接收视频编号后，由 ViewModel 请求详情，Activity 观察视频数据并将 video_file 交给播放器，同时刷新评论。简介与评论由 ViewPager2 组织，使媒体区域和信息区域在同一页面中协同展示。

MediaPlayerManager 对 ExoPlayer 进行单例封装，使用应用级 Context 创建实例，并绑定到 PlayerView。切换媒体时设置 MediaItem，完成准备后播放。页面销毁时停止播放、清空媒体并释放播放器，同时清除单例引用。

代码 4-3　媒体地址装载与播放

```
public void play(String url) {
    if (mPlayer != null) {
        MediaItem mediaItem = MediaItem.fromUri(url);
        mPlayer.setMediaItem(mediaItem);
        mPlayer.prepare();
        mPlayer.play();
    }
}
```

代码来源：feature_mediaplay/.../player/MediaPlayerManager.java（节选）。

MediaItem 描述媒体资源，setMediaItem 更换播放对象，prepare 进入准备阶段，play 表达开始播放的意图。准备与缓冲可能尚未结束，因此调用 play 不能直接作为首帧已显示或播放无卡顿的测试结论。

上述代码摘自 MediaPlayerManager.java。播放器调用简单，但稳定性取决于媒体地址、网络状态以及生命周期配合。当前 Activity 在 onStart 中允许播放，在 onPause 和 onStop 中暂停，在 onDestroy 中释放。对于短暂遮挡后只回到 onResume 的情况，仍需验证恢复策略，避免出现页面可见而视频未恢复的问题。

播放页采用 singleTask。再次点击视频卡片时，通过 onNewIntent 接收新编号并重新请求详情，复用现有页面。切换视频时不仅要更新媒体，还应核对评论、点赞数、收藏状态和推荐内容是否全部对应新视频，防止旧状态残留。

4.4.1 播放链路与资源释放

视频播放与浏览记录的数据流程如图 4-3 所示。

收到新视频编号后，应使播放器、标题、互动状态与评论指向同一视频。当前 onNewIntent 会重新请求详情并恢复评论加载标记；关联推荐列表也需要在切换后核对。详情请求成功后就写入浏览记录，因此记录表示“访问过详情”，并不证明视频已完整观看。

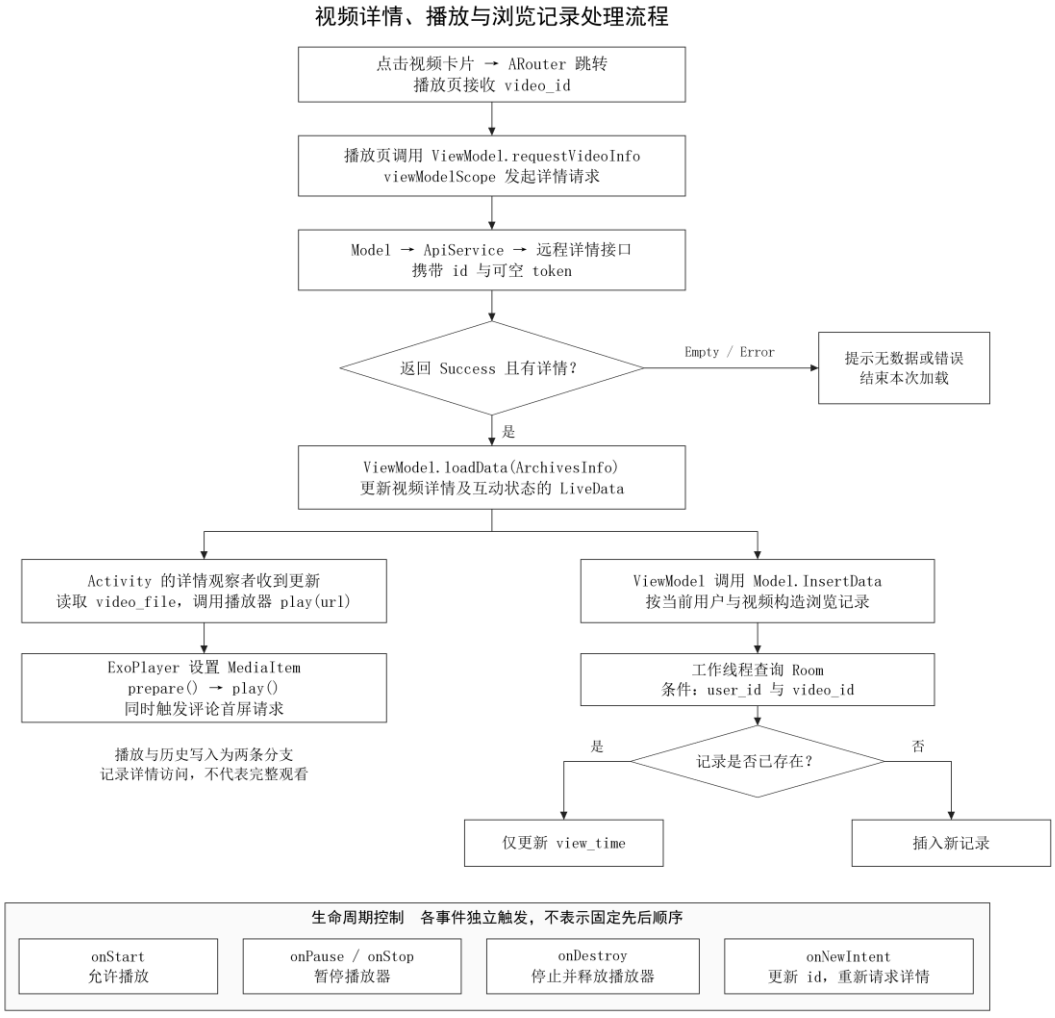


图 4-3 视频播放与浏览记录的数据流程

播放器在各生命周期入口中的处理与验证重点见表 4-2。

表 4-2　播放器生命周期处理与验证重点

生命周期入口	当前处理	验证重点
onStart	允许播放，注册事件	页面首次显示与后台返回
onPause / onStop	暂停播放	短暂遮挡与切后台
onDestroy	停止、清空媒体并释放	退出后无音频与资源残留

4.4.2 视频简介与评论界面

视频播放页的简介与评论界面如图 4-4 所示。

两张截图中的播放位置与视频内容一致，下部区域分别显示简介和评论。简介提供标题、描述、互动入口及相关内容，评论区域显示评论列表和输入入口。页面切换以同一视频详情为上下文，避免用户从简介切到评论时改变播放对象。

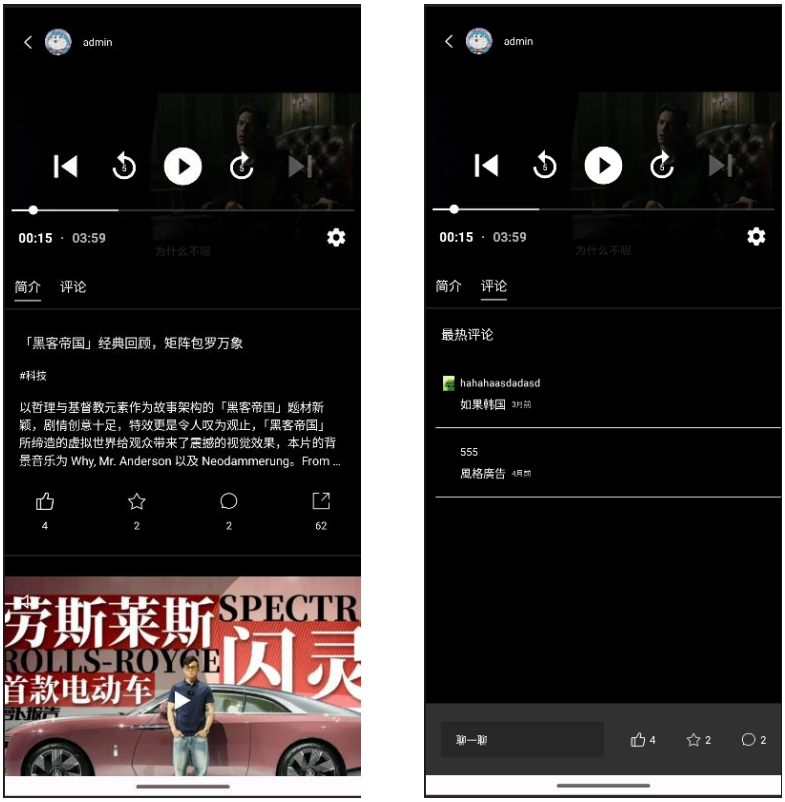


图 4-4　视频播放页的简介与评论

（a）视频简介及互动入口　　（b）同一视频的评论列表

截图可用于说明界面组织及评论展示；音视频同步、连续播放、评论提交成功和后台恢复需要实际操作记录支持。画面中的分享图标仅说明入口外观，不能据此认定分享链路已完成。

4.5 用户登录与互动状态

用户模块提供手机号验证码登录入口。登录成功后保存令牌与资料，后续受保护接口通过 token 请求头携带身份信息。退出登录清理本地会话，并通过 EventBus 通知相关页面刷新。令牌非空只表示客户端存在会话，是否有效仍需由服务端校验。

代码 4-4　视频详情接口允许游客令牌为空

```
@GET(NetAddress.VIDEO_INFO_ADDRESS)
suspend fun getVideoInfo(
    @Query("id") id: String,
    @Header("token") token: String?
): ResBase<ResVideoAllInfo>
```

代码来源：feature_mediaplay/.../apiKotlinVersion/MediaApiService.kt（节选）。

可空 token 允许游客请求公开详情；需要身份的互动操作仍在 ViewModel 中先判断登录。客户端校验用于引导操作，服务端仍应校验令牌有效性及评论等资源的操作权限。

当前 Kotlin 点赞与取消点赞在请求返回 Success 后才更新 mIsLike、mLikes 和详情对象，不属于请求前直接改变数字的处理方式。失败时应保留原状态；连续点击仍需通过请求进行中标记避免重复提交。

评论删除则先从本地列表移除，再发起删除请求，失败时可能出现列表与服务端不同步。该操作适合在成功后移除，或保存原位置并在失败时恢复。按昵称判断可删除评论只是一层界面限制，最终权限应由服务端依据用户身份决定。

4.5.1 个人中心与头像选择

个人中心与系统相册选择头像的界面如图 4-5 所示。

个人中心展示昵称或脱敏账号信息，并提供收藏、浏览记录及退出入口。右图为系统照片选择器，并非 CameraX 的相机预览；它说明资料编辑能够进入相册选择流程，尚不能单凭这一画面确认图片上传及头像更新已成功。

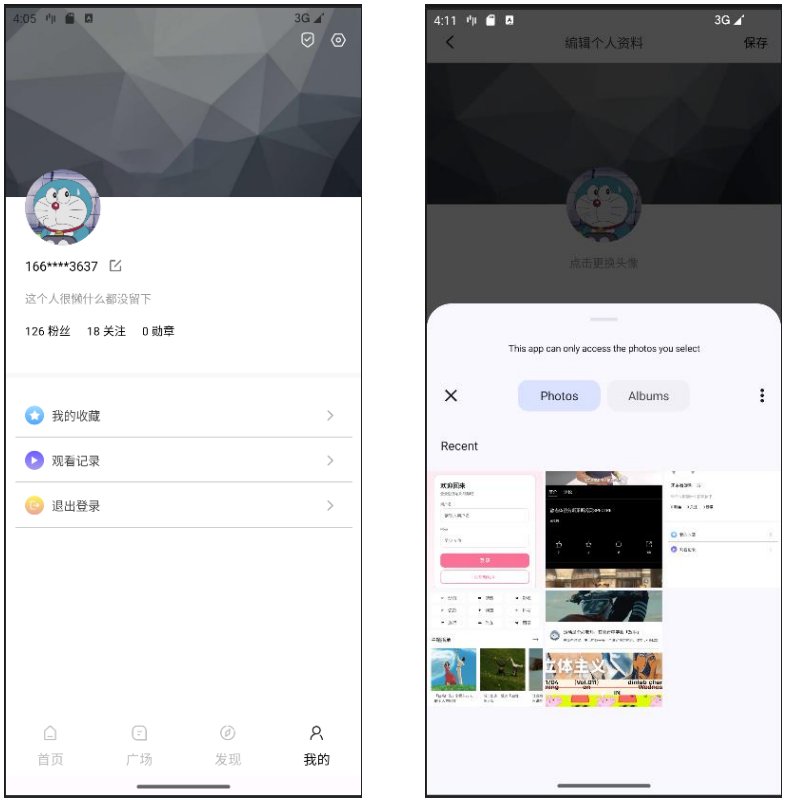


图 4-5　用户资料与头像选择界面

（a）已登录个人中心　　（b）系统相册选择头像

头像处理需分别验证拍照、相册选择、取消选择和上传失败。若使用系统照片选择器，授权范围应围绕选中的照片；相机入口则需要单独核对相机权限拒绝后的提示与返回行为。

4.6 广场与内容分类

广场将轮播内容与双列图片卡片组合展示，轮播占据完整行，其余卡片按双列排列。该设计适合突出精选内容，并保留进入分类或大图浏览的入口。

广场的轮播与双列图片布局如图 4-6 所示。

适配器使用不同条目类型处理轮播与普通图片。由于第一个位置被轮播占用，普通图片的数据索引需要减去相应偏移；点击事件同样需要使用转换后的索引。图片浏览页用横向 ViewPager2 展示组图，并通过页码提示当前查看位置。



图 4-6　广场界面

广场轮播与图片网格

发现页的分类和主题播单与广场共同构成多入口浏览。对空分类、话题数量不足以及图片加载失败应提供容错，避免直接访问固定下标导致界面异常。

4.7 本地浏览记录与数据持久化

Room 实体 UserLookRecord 对应 user_look_record 表。每次访问视频详情后，按用户编号和视频编号查找记录；已存在时更新时间，不存在时插入。读取时按浏览时间倒序排列，支持对选中视频批量删除。数据库访问在工作线程执行，结果通过主线程 Handler 返回界面。

浏览记录的主要字段、类型及作用见表 4-3。

表 4-3　浏览记录数据字段

字段	类型	作用
id	int	自增主键
user_id / video_id	String / int	区分用户及视频
cover / title / label	String	封面、标题、标签
duration	String	视频时长展示
view_time	long	浏览时间与倒序排序

代码 4-5　按用户读取并倒序展示浏览记录

```
@Query("SELECT* FROM user_look_record " +
       "WHERE user_id = :user_id " +
       "ORDER BY view_time DESC ")
List<UserLookRecord> getRecords(String user_id);
```

代码来源：library_base/.../db/UserLookRecordDao.java；长 SQL 按行拆分展示（节选）。

user_id 用于隔离账号记录，view_time 用于把最近访问的内容排在前面。插入前通过用户与视频组合查询，已有记录只更新时间。当前先查后写属于应用层控制，并发请求下仍应借助组合唯一约束与事务保证一致性。

数据库操作在工作线程执行，回调切回主线程后再更新列表。浏览记录保存的是封面、标题和视频编号等元数据，重新播放仍依赖媒体地址及网络连接。

4.7.1 收藏与历史回看界面

收藏列表与浏览记录界面如图 4-7 所示。

收藏表达用户主动保留的内容，浏览记录记录最近访问内容，二者的数据来源和业务含义不同。前者通过接口读取，后者从本机 Room 查询。相同视频可以同时出现在两个列表，不能因显示重复就将两类数据合并。

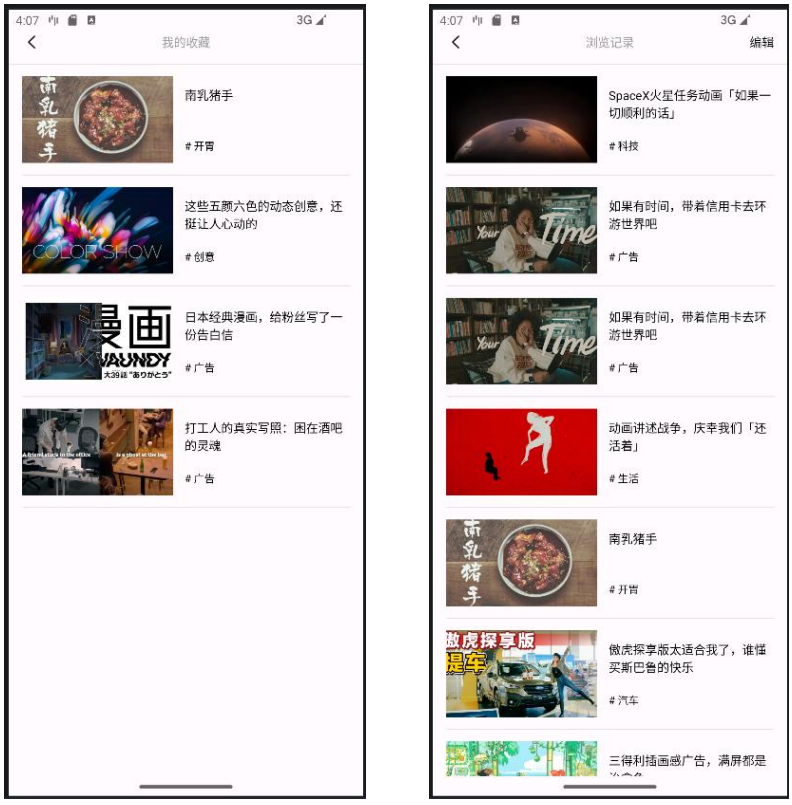


图 4-7　收藏与浏览记录

（a）远程收藏列表　　（b）本地浏览记录

浏览记录页面的编辑入口用于选择并删除记录。验证时应在删除后退出并重新进入页面，确认数据库已更新；重复查看同一视频时，应检查记录是否只更新时间，以及账号切换后是否只读取对应用户的数据。

五　实验结果与分析

5.1 功能验证与测试设计

现有截图展示了首页、发现、播放、评论、个人中心、广场、收藏、浏览记录和相册选择等页面。静态代码核对可以说明功能调用路径，但不能替代设备运行测试。以下记录以“界面可观察现象”和“待验证行为”分别说明，不将全部功能直接标为通过。

系统功能测试步骤、预期结果及当前记录状态见表 5-1。

表 5-1　系统功能测试用例与记录状态

测试项目	测试步骤与预期结果	记录状态
四入口切换	切换后返回首页，列表与位置保持	已有界面图；待操作核对
推荐与日报	刷新替换、加载追加，失败不跳页	待实测
游客播放	未登录打开视频，正常读取详情	待实测
视频切换	从相关列表选新视频，媒体和评论同步	待实测
生命周期	切后台、短暂遮挡及返回，暂停恢复正确	待实测
登录与退出	登录后资料刷新；退出后互动被拦截	已有个人页；待流程核对
点赞与收藏	新增及取消后，重进页面状态一致	待实测
评论操作	提交后可见，删除失败时状态可恢复	已有列表图；待提交核对
历史持久化	重复访问更新时间，删除后重进不恢复	已有记录图；待实测
网络与权限	断网或拒绝权限，提示并可返回重试	待实测

实测环境：设备型号_____；Android 版本_____；应用版本_____；网络条件_____；执行日期_____。结果记录应注明账号状态、具体步骤、实际现象和对应截图编号。

5.2 运行现象与结果分析

图 4-1 显示首页卡片和发现分类，图 4-4 显示同一视频的简介与评论，表明内容浏览与信息展示已经有相应界面。图 4-5 显示个人中心及系统相册入口，图 4-6 显示广场的轮播和双列布局，图 4-7 显示收藏与浏览记录。以上证据可用于讨论界面结构，不能外推为全部接口、权限和异常场景均已通过。

从架构上看，组件路由使首页可复用媒体模块中的列表和播放入口，公共列表基类减少刷新与加载逻辑的重复。MVVM 将界面状态与数据访问分开，Room

为历史回看提供本地数据基础。这些设计有助于组织功能，但其实际性能收益仍需测量，不能据此给出未经测试的耗时或内存指标。

5.3 问题分析与改进方向

当前实现中需要关注的问题及相应改进措施见表 5-2。

表 5-2　现有问题与改进措施

发现的问题	影响与改进方式
分页请求前推进页码	失败后可能跳页；成功后提交页码或失败回退
收藏数量更新表达式	正常数字未执行加减；明确括号并同步详情对象
评论先移除再请求	失败后列表失真；成功后删除或回滚
新旧播放状态并存	部分弹窗与 Kotlin 状态源不一致；统一页面共享状态
生命周期恢复入口	短暂遮挡后需核对恢复；尊重用户主动暂停状态
浏览记录先查后写	并发存在重复风险；组合唯一约束与事务

5.3.1 收藏数量更新的代码分析

代码 5-1　当前收藏数量的更新表达式

```
val newCollections = mCollections.value?.toInt()
    ?: ERRORCOLLECTIONS + 1
```

代码来源：MediaPlayViewModelKotlinVersion.kt；按原表达式换行（节选）。

该表达式中，加一作用于 Elvis 运算符右侧的后备值。若 mCollections.value 能正常转换为整数，结果直接取该整数，不会执行加一；取消收藏的相应表达式也有同类问题。这会使收藏状态已变化而显示数量未变化。

代码 5-2　建议的数值更新方式

```
val current = mCollections.value?.toIntOrNull() ?: 0
val updated = current + 1
mCollections.value = updated.toString()
mArchivesInfo.value?.collection = updated
```

代码来源：改进示例，尚未替换工程实现（节选）。

建议仅在接口确认成功后执行数量更新，取消收藏则减一并限制不小于零。若服务端可直接返回最新计数，应优先采用服务端值。连续点击时还应禁用按钮或串行处理，防止重复请求导致状态与数量不一致。

5.4 需要补充的运行证据

建议补充三组截图：脱敏后的验证码登录页与登录成功状态；同一视频点赞或收藏前后及重新进入页面后的对照；浏览记录删除前后与重新进入后的对照。若展示相机能力，再补一张 CameraX 预览或拍照结果图。

建议再记录一次断网后的错误提示及重试结果，并填写测试设备、Android 版本和执行日期。截图中的完整手机号、验证码和令牌应隐藏，现有相册截图不能代替相机截图。

六　小结与心得体会

本课程设计围绕 Android 短视频客户端，将内容列表、媒体播放、登录互动和本地记录组织在同一工程中。通过梳理需求与代码，我对智能终端应用的理解从单一页面控件扩展到页面状态、数据访问和资源管理之间的协作。一个页面能够显示数据只是起点，切换页面、账号变化和网络失败之后仍保持一致，才是完整应用需要解决的问题。

在架构方面，组件划分帮助我建立了职责意识。首页负责组织推荐与日报，媒体模块承担播放和互动，公共层提供路由、基类及用户管理。这样的边界使查找功能更清楚，也让我认识到公共封装应服务于真实复用；若把所有业务逻辑放入基类，反而会增加理解和修改成本。

在数据处理方面，网络响应、界面状态与本地存储各有不同的生命周期。LiveData 能通知界面刷新，但不能替代服务端确认；加密偏好存储能保护本地会话资料，但令牌是否有效仍由接口判断；浏览记录能保存回看入口，却不意味着媒体已经离线保存。明确这些区别，有助于避免功能说明与实际能力不一致。

视频播放让我更加关注生命周期。播放器准备、暂停、恢复与释放需要与页面行为配合，singleTask 复用页面时还必须同步更新关联数据。后续我会通过连续切换视频、后台返回和异常媒体地址等场景验证实现，而不只观察一次正常播放。

本次报告整理也让我认识到测试记录的重要性。功能存在于代码中与功能在设备上稳定运行是两个层面，实验结果应由截图、操作步骤和实际现象支撑。今后的学习中，我会先把核心流程拆成可验证的小任务，及时记录问题与修改依据，再通过复测确认改动效果，逐步提高开发与技术表达能力。

后续完善将优先处理分页、收藏计数和评论删除的一致性，再开展生命周期与账号切换回归测试。相比单纯增加页面，先保证核心流程在失败、返回和重复操作后仍能正确工作，更有助于形成可靠的移动应用开发习惯。